

LABORATORIO 3.5

Control remoto con sockets y joystick

Preparado por:
Luisdavid Colina V24766381
Samantha Ramírez V31307714

1. Desarrollo de actividades

1.1. Actividad 6.1: Servidor de control remoto (socket server)

Enunciado: Implemente un código de Processing que actúe como servidor TCP. El servidor debe recibir comandos de ángulos enviados desde un cliente remoto y retransmitirlos al servo físico.

Processing trae incluida la librería `processing.net`, que permite montar servidores y clientes TCP sin instalar nada extra [2]. Para esta actividad se creó un objeto `Server` escuchando en el puerto 12345 (Código 1), mientras que el servo queda configurado en el pin 3 en modo `Arduino.SERVO` con una posición inicial de 90°.

Código 1: Inicialización del servidor TCP y del servo en `setup()`.

```
import processing.serial.*;
import processing.net.*;
import cc.arduino.*;

Arduino arduino;
Server miServidor;
int servoPin = 3;
int anguloServo = 90;

arduino = new Arduino(this, Arduino.list()[2], 57600);
arduino.pinMode(servoPin, Arduino.SERVO);
arduino.servoWrite(servoPin, anguloServo);

miServidor = new Server(this, 12345);
```

Lo primero que se hizo fue armar el circuito conectando el servomotor a uno de los pines digitales con soporte PWM de la placa, tal como se ve en la Figura 1.

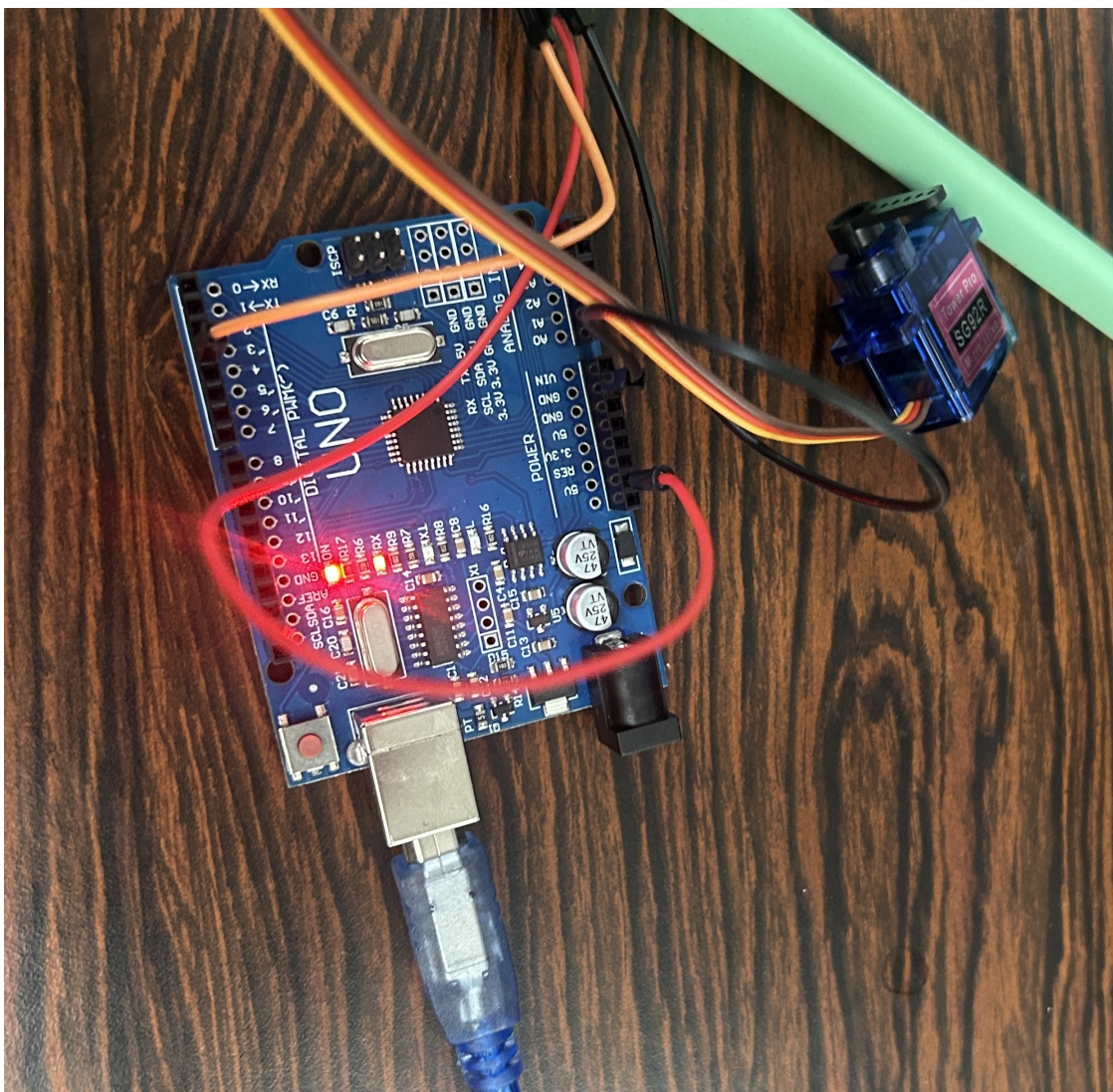


Figura 1: Conexión del servomotor al pin digital del microcontrolador.

En `draw()`, el servidor revisa si hay algún cliente con datos disponibles usando `miServidor.available()`. Cuando los hay, lee la cadena hasta encontrar el carácter `'\n'` que el cliente agrega al final de cada comando. Ese delimitador es necesario porque TCP no tiene bordes de mensaje, sin él no habría forma de saber dónde termina un valor y dónde empieza el siguiente (Código 2).

Validar que el valor esté en el rango $[0, 180]$ antes de llamar a `servoWrite()` es importante para proteger el hardware, un valor fuera de ese rango puede dañar el servo mecánicamente o producir una señal PWM incorrecta.

Código 2: Recepción, validación y aplicación del ángulo en draw().

```
Client clienteEntrante = miServidor.available();

if (clienteEntrante != null) {
    String cadena = clienteEntrante.readStringUntil('\n');

    if (cadena != null) {
        cadena = trim(cadena);
        try {
            int nuevoAngulo = int(cadena);
            if (nuevoAngulo >= 0 && nuevoAngulo <= 180) {
                anguloServo = nuevoAngulo;
                arduino.servoWrite(servoPin, anguloServo);
            }
        } catch (Exception e) {
            println("Error de trama: " + e.getMessage());
        }
    }
}
```

En la Figura 2 se puede ver el servo siendo controlado desde otra máquina distinta a la que tenía el Arduino. Otro grupo actuó como consola de control con la interfaz gráfica, mientras el hardware respondía en tiempo real



Figura 2: Servo siendo controlado por otro grupo (cliente).

1.2. Actividad 6.2: Cliente remoto con interfaz gráfica (socket client)

Enunciado: Desarrolle un cliente TCP en Processing que permita controlar remotamente el servo del servidor.

Para actuar como clientes, se usó `controlP5` para los controles gráficos y `processing.net` para la conexión TCP. Al arrancar, se creó un objeto `Client` apuntando a la IP del servidor (la IP del otro grupo) y al puerto que el router tenía redirigido (Código 3).

Código 3: Creación del cliente TCP y los controles en `setup()`.

```
import controlP5.*;
import processing.net.*;

Client miCliente;
ControlP5 cp5;
int anguloActual = 0;

// IP del equipo con el Arduino. Puerto: 8080
miCliente = new Client(this, "190.169.74.67", 8080);

cp5.addSlider("slider1").setPosition(50, 50).setRange(0, 255).setSize(150, 20);
cp5.addSlider("slider2").setPosition(50, 110).setRange(0, 255).setSize(150, 20);
cp5.addSlider("slider3").setPosition(50, 170).setRange(0, 255).setSize(150, 20);

cp5.addKnob("Servo").setPosition(500, 60).setRadius(30).setRange(0, 180).setValue(90);
cp5.addKnob("knob2").setPosition(500, 150).setRadius(30).setRange(0, 255);
cp5.addKnob("knob3").setPosition(500, 240).setRadius(30).setRange(0, 255);
```

El envío de datos al servidor ocurre únicamente desde el callback `controlEvent()`, que `ControlP5` llama automáticamente cada vez que el usuario mueve un control. Solo el knob "Servo" genera tráfico de red; los demás controles son puramente visuales (Código 4). La comprobación de `miCliente.active()` evita que el programa falle si la conexión se cae.

Código 4: Envío del ángulo al servidor en `controlEvent()`.

```
void controlEvent(ControlEvent elEvento) {
    if (elEvento.isController()) {
        if (elEvento.getController().getName().equals("Servo")) {
            anguloActual = (int)elEvento.getController().getValue();
            if (miCliente.active()) {
                miCliente.write(anguloActual + "\n");
            }
        }
    }
}
```

En `draw()` se llama a `dibujarServo()` para animar el brazo del servo en pantalla con el ángulo actual (Código 5).

Código 5: Renderizado de brazo del servo en `draw()`.

```
void draw() {
    background(30);
    dibujarServo(anguloActual);
}

void dibujarServo(int angulo) {
    pushMatrix();
    translate(450, 90);
    rotate(radians(angulo));
    stroke(0, 255, 0);
    strokeWeight(4);
    line(0, 0, 40, 0);
    fill(100);
    ellipse(0, 0, 15, 15);
    popMatrix();
}
```

En la Figura 3 y Figura 4 se confirmó que el servo del otro grupo respondía físicamente de forma precisa a los comandos llegados por red.

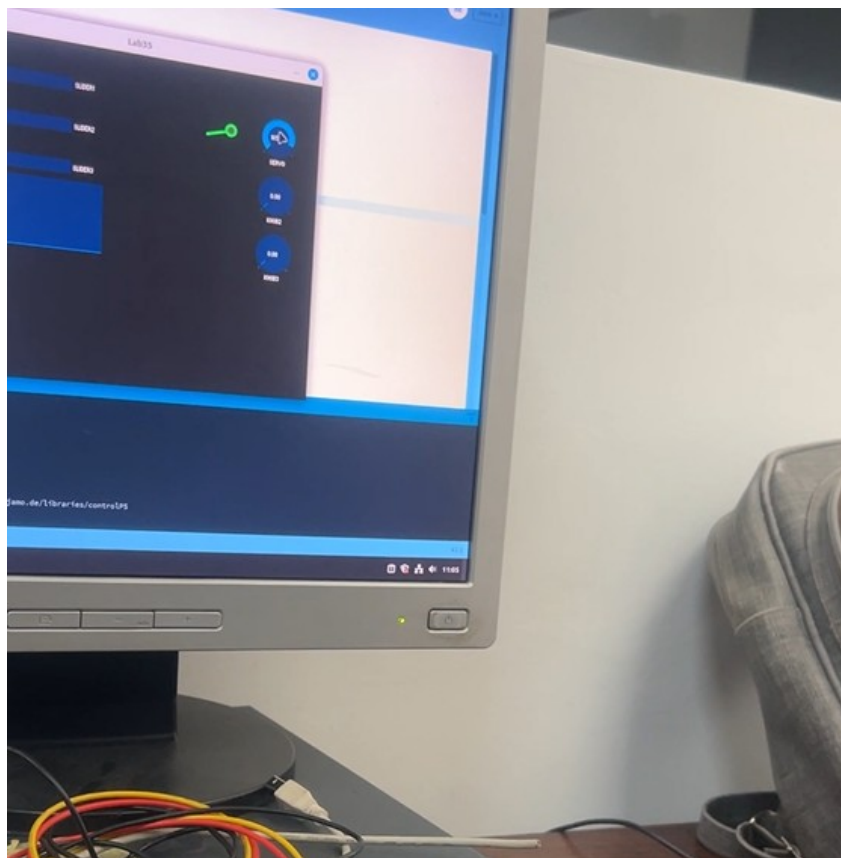


Figura 3: Cliente operando el servo de forma remota.

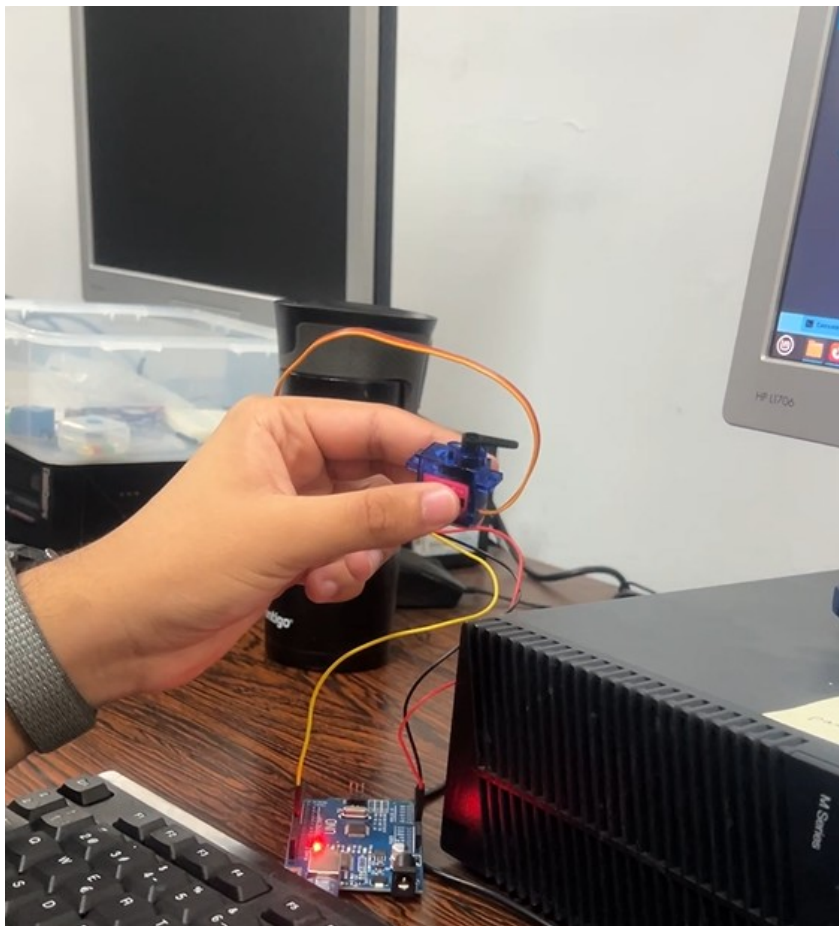


Figura 4: Servo de otro grupo siendo operado por el cliente.

Finalmente, en la Figura 5 se visualizan las interfaces para el cliente y el servidor.

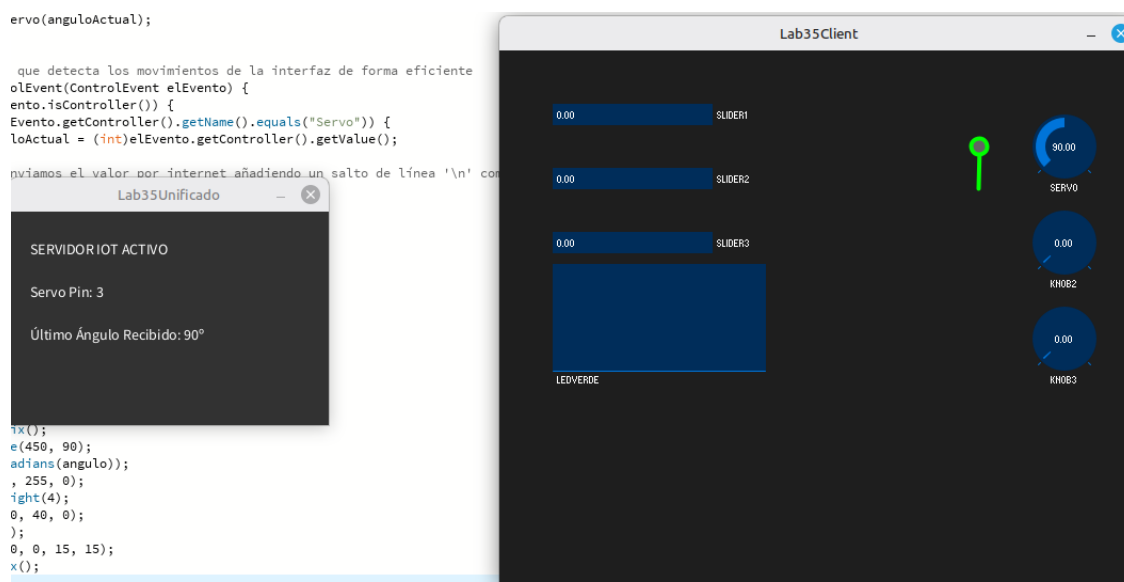


Figura 5: Interfaz de Socket Server y Socket Client.

1.3. Actividad 6.3: Interfaz unificada con joystick y servomotor

Enunciado: Desarrolle una aplicación en Processing que integre la lectura física de un módulo joystick analógico con el control de un servomotor. El eje X mapeará ángulos de 0° a 90° y el eje Y de 90° a 180° . La interfaz gráfica debe mostrar los valores ADC de cada eje, el estado del pulsador y una representación visual dinámica del brazo del servo.

El joystick KY-023 tiene tres señales: dos analógicas de posición (VRX y VRY) y un pulsador digital (SW). Las primeras dos se conectaron a las entradas A0 y A1 respectivamente, y el pulsador al pin digital 2. El servo fue al pin 3, que Firmata maneja como salida de posicionamiento [1]. El montaje completo se muestra en la Figura 6.

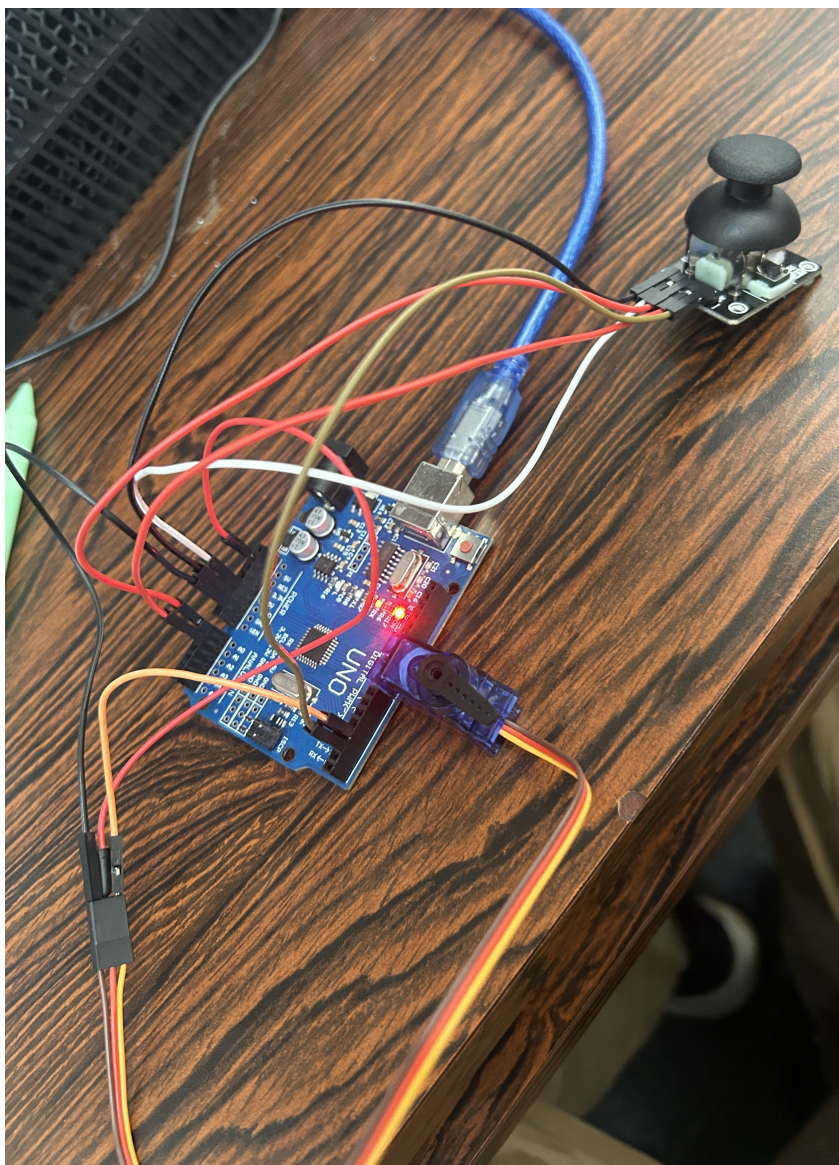


Figura 6: Montaje final unificado con Joystick y Servomotor sobre protoboard.

La declaración de pines y la configuración inicial en `setup()` se puede ver en el Código 6:

Código 6: Declaración de pines y configuración en `setup()` del sketch unificado.

```
int pinVRX = 0; // Entrada analogica A0 (Controla 0 a 90 grados)
int pinVRY = 1; // Entrada analogica A1 (Controla 90 a 180 grados)
int pinSW = 2;  // Entrada digital Pin 2 (Pulsador)
int servoPin = 3; // Pin de control del servo motor

arduino = new Arduino(this, Arduino.list()[2], 57600);
arduino.pinMode(pinSW, Arduino.INPUT);
arduino.digitalWrite(pinSW, Arduino.HIGH);
arduino.pinMode(servoPin, Arduino.SERVO);
arduino.servoWrite(servoPin, 90);      // Posicion inicial segura
```

Dentro de `draw()` se leen los tres valores del hardware y luego se compara cuál de los dos ejes se aleja más del centro de reposo (512 en un ADC de 10 bits) y ese es el que controla el servo. Si gana el eje X, el valor se mapea al rango $[0^\circ, 90^\circ]$; si gana el eje Y, al rango $[90^\circ, 180^\circ]$ (Código 7). De esa forma se evita que ambos ejes compitan y el servo reciba señales contradictorias.

Código 7: Lectura del joystick y lógica de eje dominante en `draw()`.

```
valorX = arduino.analogRead(pinVRX);
valorY = arduino.analogRead(pinVRY);
estadoSW = arduino.digitalRead(pinSW);

if (abs(valorX - 512) > abs(valorY - 512)) {
    // Eje X domina: rango 0 a 90 grados
    anguloServo = (int) map(valorX, 0, 1023, 0, 90);
} else {
    // Eje Y domina: rango 90 a 180 grados
    anguloServo = (int) map(valorY, 0, 1023, 90, 180);
}
arduino.servoWrite(servoPin, anguloServo);
```

El comportamiento al mover el joystick hacia la derecha (eje X positivo, rango 0° a 90°) se puede ver en la Figura 7.

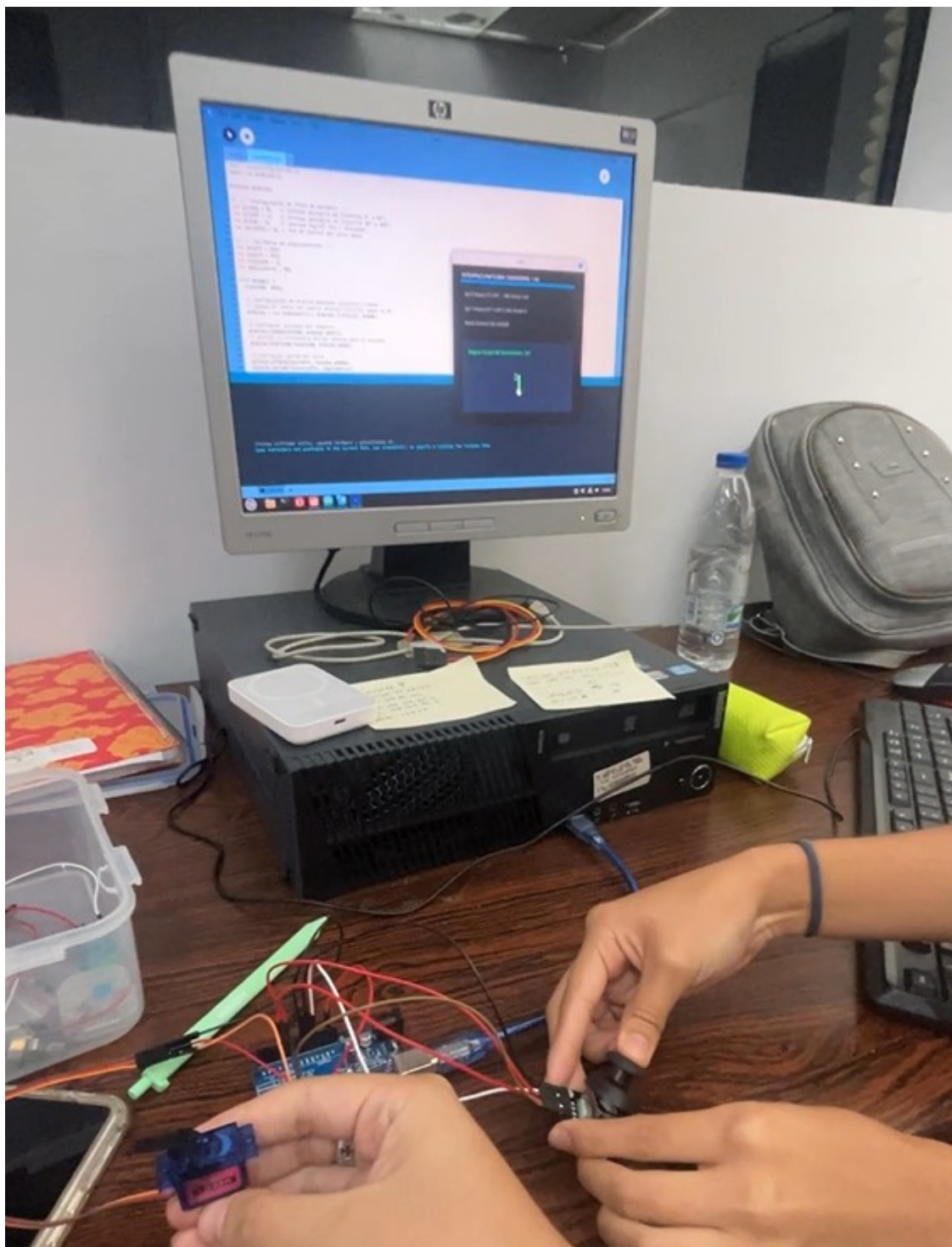


Figura 7: Prueba de sistema: Respuesta al movimiento positivo en el eje X.

La interfaz también muestra en pantalla los valores ADC de cada eje y el estado del pulsador. La función `dibujarServoGrafico()` dibuja el brazo animado (Código 8).

Código 8: Función auxiliar para dibujar el brazo del servo en pantalla.

```
void dibujarServoGrafico(int x, int y, int angulo) {  
    pushMatrix();  
    translate(x, y);  
    rotate(radians(angulo - 90)); // 90 deg apunta arriba en pantalla  
    stroke(0, 255, 100);  
    strokeWeight(6);  
    line(0, 0, 0, -50); // Brazo mecanico  
    fill(255);  
    noStroke();  
    ellipse(0, 0, 15, 15); // Eje central circular  
    popMatrix();  
}
```

El `delay(20)` al final de `draw()` introduce una pequeña pausa para estabilizar la comunicación con Firmata y evitar que lecturas muy rápidas saturen el puerto serial [1]. En la Figura 8 se puede ver la interfaz completa con todos los datos del ADC y el ángulo del servo actualizándose en tiempo real.

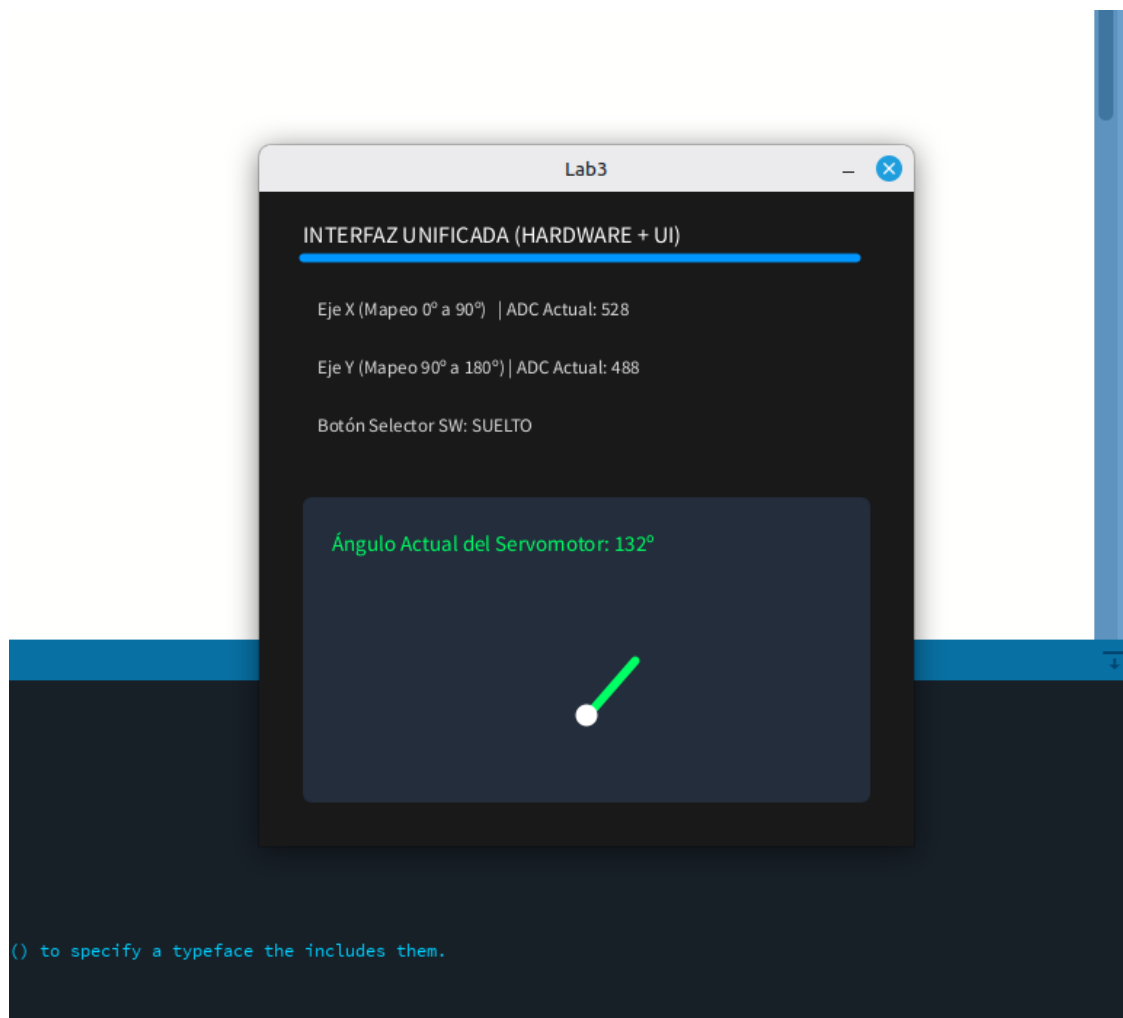


Figura 8: Interfaz de usuario final con datos del ADC y ángulo del servo en tiempo real.

2. Conclusiones

En este laboratorio se trabajó en extender el control de hardware desde una interfaz local hacia una arquitectura cliente-servidor usando sockets TCP.

En la actividad 6.1 quedó claro que TCP entrega los datos en orden y sin pérdidas, pero no separa los mensajes por sí solo. Usar `'\n'` como delimitador junto con `readStringUntil()` fue la forma más simple de reconstruir cada comando en el receptor. Además, validar el rango antes de escribir al servo resultó clave para no dañar el hardware.

En la actividad 6.2 se vio la ventaja de usar el callback `controlEvent()` en lugar de leer el estado de los controles en cada fotograma de `draw()`: solo se envían datos por red cuando el usuario realmente mueve el knob del servo.

En la actividad 6.3 se comprobó que la lógica de eje dominante es una solución sencilla y efectiva para el problema de interferencia entre los dos ejes del joystick. Al quedarse solo con el eje que más se aleja del centro, el control resulta intuitivo y predecible.

Referencias

- [1] F. Mellis, H. Barragan y otros, “Firmata Protocol Documentation,” *Firmata GitHub Repository*, 2006–2024. [En línea]. Disponible: <https://github.com/firmata/protocol>. [Consulta: jun. 2026]
- [2] The Processing Foundation, “processing/net — Network Library,” *Processing Reference*, 2001–2024. [En línea]. Disponible: <https://processing.org/reference/libraries/net/>. [Consulta: jun. 2026]
- [3] A. Cockpit, “controlP5 — A GUI Library for Processing,” 2012–2023. [En línea]. Disponible: <https://www.sojamo.de/libraries/controlP5/>. [Consulta: jun. 2026]
- [4] Arduino, “Arduino Language Reference,” *Arduino Documentation*, Arduino LLC, 2024. [En línea]. Disponible: <https://docs.arduino.cc/language-reference/>. [Consulta: jun. 2026]
- [5] A. Russoniello, “Laboratorio: Comunicación TCP y Control Remoto con Processing,” guía de práctica de laboratorio, Laboratorio General (Internet de las Cosas), Escuela de Computación, Facultad de Ciencias, Universidad Central de Venezuela, Caracas, 2026.